

ВВЕДЕНИЕ

Автоформализация математических текстов представляет собой задачу преобразования утверждений, записанных на естественном языке, в структурированный вид, пригодный для дальнейшего анализа и представления на формальном языке. Для обучения и оценки соответствующих моделей требуются датасеты, содержащие исходные математические формулировки, их тематические признаки, формальные представления и заготовки кода для систем доказательства теорем.

Большинство существующих ресурсов в области формализации математики ориентировано преимущественно на англоязычные тексты. В связи с этим актуальной является подготовка русскоязычных данных, извлечённых из математических публикаций. В данной работе рассматриваются высказывания об алгебраических структурах: группах, кольцах, полях, модулях, алгебрах, идеалах, гомоморфизмах и изоморфизмах.

В ходе работы разработана программа на языке Python, предназначенная для обработки PDF-файлов математических статей, извлечения текстовой информации, поиска предложений с алгебраическим содержанием и формирования структурированного датасета. Программа не выполняет полную строгую формализацию математических теорем, а формирует основу датасета: исходные русскоязычные фрагменты, предварительные англоязычные представления, классификацию утверждений, формальные шаблоны и заготовки Lean 4 + Mathlib.

1 ПОСТАНОВКА ЗАДАЧИ

Целью работы является разработка программного средства для подготовки обучающего датасета по теме «Высказывания об алгебраических структурах» на основе русскоязычных PDF-публикаций.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать извлечение текста из PDF-файлов математических статей.
2. Выполнить очистку текста от артефактов извлечения, служебных фрагментов начала публикации и списка литературы в конце документа.

3. Сформировать глоссарий основных алгебраических объектов и таблицу встречающихся обозначений.
4. Реализовать поиск фрагментов текста, содержащих математические утверждения по выбранной тематике.
5. Добавить контекст к найденным предложениям, сохранив соседние предложения до и после основного утверждения.
6. Классифицировать извлечённые записи по категориям **ISOMORPH**, **SUBSTRUCTURE**, **ACTION**, **CLASSIFICATION**, **PROPERTY**.
7. Сформировать поля датасета: русскоязычную формулировку, предварительное англоязычное представление, формальную запись, заготовку Lean 4 + Mathlib, ключевые слова, тип утверждения и источник.
8. Организовать сохранение результатов в форматах JSON, CSV и XLSX.
9. Подготовить файл для ручной аннотации и расчёта межаннотаторского согласия с использованием коэффициента Cohen's kappa.

Результатом работы должна стать программа, позволяющая преобразовать набор PDF-файлов в структурированные файлы датасета, пригодные для дальнейшей ручной проверки, разметки и использования в задачах автоматизированной формализации математического текста.

2 ИСПОЛЬЗУЕМЫЕ СРЕДСТВА РАЗРАБОТКИ

Для реализации программы использован язык программирования Python. Выбор Python обусловлен наличием библиотек для обработки PDF-документов, работы с текстом и формирования табличных файлов.

Для извлечения содержимого PDF-файлов применяется библиотека PyMuPDF, подключаемая в программе через модуль `fitz`. Она позволяет открывать PDF-документы, проходить по страницам и извлекать текстовые блоки вместе с информацией о странице и координатах.

Для обработки текста используются стандартные средства Python и модуль `re`, обеспечивающий работу с регулярными выражениями. Регулярные выражения применяются для поиска терминов, математических конструкций, маркеров утверждений и удаления нежелательных фрагментов текста.

Модули `json` и `csv` используются для сохранения структурированных результатов. Библиотека `pandas` применяется для представления итоговых записей в табличном виде, а `openpyxl` — для сохранения и оформления Excel-файла `Dataset.xlsx`.

Для создания формальных заготовок используется синтаксис Lean 4 и математическая библиотека Mathlib. В рамках данной работы Lean-код формируется как шаблон, соответствующий тематическому типу утверждения, и предназначен для дальнейшей ручной проверки и уточнения.

3 СТРУКТУРА ФОРМИРУЕМОГО ДАТАСЕТА

Итоговая запись датасета содержит семь полей.

Поле «Формулировка на естественном языке (русский)» хранит извлечённый из публикации фрагмент. В него включается основное найденное предложение и, при наличии, одно предложение до него и одно предложение после него. Такой подход позволяет сохранить контекст математического высказывания и снизить вероятность потери условий утверждения.

Поле «Формулировка на естественном языке (английский)» содержит предварительное англоязычное представление записи. В текущей реализации оно формируется с помощью словарной замены основных математических терминов и конструкций. Следовательно, это поле является заготовкой для последующей ручной проверки, а не полноценным автоматическим переводом математического текста.

Поле «Запись на формальном языке» содержит шаблон логической записи, выбираемый в зависимости от классифицированного типа утверждения. Например, для записей, связанных с изоморфизмом, формируется конструкция с отношением $\cong$, а для записей о подструктурах — шаблон с включением подгруппы, подмодуля или идеала.

Поле «Код Lean 4 + Mathlib» содержит шаблон теоремы на языке Lean. Данный код не является автоматическим доказательством исходного утверждения из статьи, а показывает возможную форму записи объекта или свойства в среде формального доказательства.

Поле «Ключевые слова (русский / английский)» хранит найденные тематические признаки: группа, подгруппа, кольцо, поле, модуль, изоморфизм, гомоморфизм и другие.

Поле «Тип утверждения» содержит категорию, назначенную программой на основе найденных терминов и регулярных выражений. Предусмотрены следующие типы:

- **ISOMORPH** — утверждения об изоморфизмах и эквивалентности структур;
- **SUBSTRUCTURE** — утверждения о подгруппах, идеалах, подмодулях и других вложенных структурах;
- **ACTION** — утверждения о действиях групп, орбитах и стабилизаторах;
- **CLASSIFICATION** — утверждения, относящиеся к классификации объектов;
- **PROPERTY** — утверждения о свойствах алгебраических структур.

Поле «Источник» содержит имя текстового файла, полученного из исходной PDF-публикации. Это позволяет установить происхождение каждой записи и при необходимости обратиться к первичному документу.

4 ОПИСАНИЕ РЕАЛИЗАЦИИ ПРОГРАММЫ

4.1 Исходные справочные структуры

В начале программы задаётся словарь **GLOSSARY**. Он содержит основные понятия выбранной предметной области: группу, подгруппу, нормальную подгруппу, кольцо, идеал, поле, векторное пространство, модуль, алгебру, гомоморфизм и изоморфизм.

Для каждого объекта указываются определение, примеры и характерные обозначения. Данный словарь нужен для формирования самостоятельного справочного файла `glossary.json`, а также для представления предметной области, в рамках которой формируется датасет.

Словарь **KEY_TERMS** содержит регулярные выражения для поиска терминов в русскоязычном и англоязычном тексте. Например, для понятия «группа»

предусмотрен поиск словоформ русского слова и английского слова `group`. Использование регулярных выражений позволяет обнаруживать термины в разных падежах и формах.

Словарь `NOTATION_PATTERNS` предназначен для поиска типичных математических обозначений. Для групп анализируются символы `G`, `H`, `K`, для колец — `R`, `S`, для полей — `F`, `K`. На основе найденных обозначений программа формирует статистическую таблицу их использования в обрабатываемых статьях.

Массив `DATASET_FIELDS` определяет единую структуру итогового датасета. Он используется при создании CSV- и XLSX-файлов и гарантирует одинаковый порядок столбцов во всех экспортируемых представлениях.

4.2 Предобработка текстовых данных

Основная логика очистки текста реализована в классе `TextPreprocessor`. Выделение предобработки в отдельный класс необходимо, поскольку одинаковые операции используются на нескольких этапах программы: при построении глоссария, подсчёте статистики и формировании записей датасета.

Метод `clean_text()` выполняет базовую нормализацию извлечённого текста. Из строки удаляются мягкие переносы, заменяются специальные символы лигатур, устраняются разрывы слов, возникшие из-за переноса строки в PDF, и нормализуются последовательности пробелов.

Дополнительно в программе предусмотрено удаление управляющих символов, недопустимых для записи в Excel. Такая очистка необходима, поскольку при извлечении содержимого PDF в строку могут попадать невидимые символы. Они не видны пользователю при чтении текста, но вызывают ошибку при сохранении данных библиотекой `openpyxl`.

Метод `remove_front_and_back_matter()` исключает части статьи, которые не должны попадать в извлекаемый датасет. В начале документа выполняется поиск служебных элементов, таких как аннотация, `abstract`, введение или начало первого раздела. В конце текста ищутся маркеры списка литературы: «Литература», «Список литературы», `References`, `Bibliography`. Это позволяет уменьшить количество ложных записей,

полученных из названия статьи, сведений об авторах и библиографических ссылок.

Метод `split_sentences()` разделяет очищенный текст на предложения. В обработку включаются только фрагменты допустимой длины: слишком короткие строки обычно не содержат полноценного математического смысла, а чрезмерно длинные фрагменты часто являются результатом некорректного извлечения текста из PDF.

4.3 Обработка путей проекта

В программе реализован класс `PathResolver`. Он обеспечивает корректное определение папок с исходными и выходными файлами.

Необходимость данного класса связана с тем, что программа может запускаться разными способами: из корневой папки проекта либо из папки, в которой находится сам файл `main.py`. Если пути обрабатываются только относительно текущей директории PowerShell, программа может не обнаружить папку `pdfs` или сохранить результаты в неожиданное место.

Методы класса определяют путь к каталогу с PDF-файлами и путь к каталогу `output`. Благодаря этому программа может использоваться более устойчиво на разных компьютерах и при различных вариантах запуска.

4.4 Построение глоссария и таблицы обозначений

Класс `GlossaryBuilder` отвечает за формирование справочной части результатов.

Метод `extract_text_with_context()` открывает PDF-файл, проходит по его страницам и получает текстовые блоки. Для каждого блока сохраняются номер страницы, координаты блока и извлечённый текст. Информация о странице и расположении текста может использоваться для контроля обнаруженных терминов в исходной публикации.

Метод `find_object_examples()` ищет употребления каждого понятия из глоссария в извлечённом тексте. Для найденного совпадения сохраняется небольшой фрагмент окружающего контекста. Благодаря этому можно проверить, действительно ли термин использован в математическом смысле.

Метод `find_notations()` подсчитывает встречаемость характерных обозначений математических объектов. На основе результата формируется таблица со столбцами «тип объекта», «примеры обозначений» и «число упоминаний».

Метод `build_table_from_pdfs()` объединяет обработку всех PDF-файлов. В результате создаются файлы `glossary.json` и `notation_table.csv`. Первый файл содержит определения и примеры терминов, второй — статистику обозначений, найденных в публикациях.

4.5 Извлечение текста и статистика терминов

Класс `ArticleStatsExtractor` предназначен для обработки исходных статей и получения подготовленных текстовых файлов.

Метод `extract_text_from_pdf()` получает текст каждой страницы PDF-документа. После этого метод `postprocess_text()` применяет очистку и удаляет служебные части статьи.

Для каждого PDF-файла формируется отдельный очищенный текстовый файл в каталоге `output/txt_clean`. Такое сохранение необходимо по двум причинам. Во-первых, оно позволяет просмотреть результат извлечения до формирования датасета. Во-вторых, следующий этап программы может работать уже с очищенными текстами, не открывая PDF повторно.

Метод `count_terms()` подсчитывает количество упоминаний ключевых понятий. В статистику включаются как основные объекты, например группа, кольцо или поле, так и целевые фрагменты слов: «изоморф», «гомоморф», «подгрупп». Это позволяет оценить тематическую пригодность выбранных публикаций.

Общая статистика по статьям сохраняется в файл `article_stats.csv`. С помощью данного файла можно определить, какие публикации содержат больше высказываний по выбранной теме и подходят для наполнения датасета.

4.6 Извлечение математических утверждений

Главным этапом программы является класс `StatementDatasetBuilder`. Он отвечает за превращение очищенного текста в набор структурированных записей.

Метод `detect_keywords()` определяет, какие алгебраические термины встречаются в конкретном текстовом фрагменте. Если предложение не содержит ни одного термина предметной области, оно не рассматривается как потенциальная запись датасета.

Метод `has_statement_marker()` проверяет наличие слов, характерных для математических утверждений: «теорема», «лемма», «следствие», «предложение», «определение», `theorem`, `lemma`, `definition` и других. Такие маркеры позволяют выделить фрагменты, которые с высокой вероятностью являются содержательными математическими конструкциями.

Метод `is_statement_candidate()` дополнительно анализирует логические слова и символы: «если», «то», «для любого», «существует», а также обозначения $\forall$, $\exists$, $\Rightarrow$, $\Leftrightarrow$, $\cong$, $\leq$, $\subseteq$. Предложение включается в список кандидатов, если содержит термин предметной области и признаки утверждения.

Для сохранения смысловой полноты используется метод `make_context_statement()`. Он добавляет к найденному предложению одно предыдущее и одно последующее предложение, если они существуют и не превышают допустимый размер. Такой подход необходим, поскольку математическое утверждение может ссылаться на объект или условие, сформулированное непосредственно перед ним.

После извлечения применяется метод `remove_duplicates()`. Он формирует хеш нормализованного основного предложения и исключает повторяющиеся записи. Удаление дублей особенно важно при обработке нескольких публикаций или при наличии повторяющихся фрагментов внутри одного документа.

4.7 Классификация утверждений

Метод `classify_statement()` выполняет автоматическую тематическую классификацию каждого найденного фрагмента.

Для каждого типа утверждения задан набор характерных слов. Например, слова «изоморф», `isomorph` и «эквивалент» повышают вероятность категории `ISOMORPH`. Термины «подгруппа», «идеал», «подмодуль» относятся к категории `SUBSTRUCTURE`. Слова «действие», «орбита», «стабилизатор» используются для определения категории `ACTION`.

Программа подсчитывает число совпадений для каждой категории и выбирает тип с максимальным значением. Если выраженных признаков не найдено, утверждение относится к общему типу `PROPERTY`.

Данный механизм является эвристическим. Он позволяет быстро выполнить первичную разметку большого количества фрагментов, однако не заменяет ручную проверку, поскольку один математический текст может одновременно относиться к нескольким смысловым категориям.

4.8 Формирование англоязычного, формального и Lean-представления

Метод `translate_to_english()` формирует предварительную англоязычную запись. В программе используется набор замен наиболее важных математических терминов и логических конструкций: «группа» заменяется на `group`, «кольцо» — на `ring`, «изоморфизм» — на `isomorphism`, «если» — на `if` и т. д.

Такой способ не является полноценным машинным переводом и не обеспечивает корректного перевода всех грамматических конструкций. Его назначение состоит в подготовке чернового параллельного представления, которое впоследствии может быть уточнено вручную.

Метод `make_formal_statement()` создаёт шаблон формальной записи в зависимости от назначенного типа. Например, для категории `ISOMORPH` используется выражение, включающее отношение изоморфизма, для `ACTION` — соотношения действия группы, а для `SUBSTRUCTURE` — шаблоны для подгрупп, подмодулей или идеалов.

Метод `make_lean_code()` формирует заготовку теоремы на языке Lean 4 с подключением библиотеки `Mathlib`. Генерируемый код отражает общую категорию найденного текста. Например, для изоморфизмов создаётся заготовка с мультипликативной эквивалентностью групп, а для подгрупп — простая теорема о включении подгруппы в саму себя.

Важно учитывать, что формальная запись и Lean-код являются демонстрационными шаблонами, а не точной автоматической формализацией исходного предложения. Для получения корректного формального корпуса требуется ручная проверка и редактирование каждой записи.

4.9 Сохранение результатов

Метод `build_dataset()` читает все очищенные текстовые файлы, извлекает кандидатов, удаляет дубли, сортирует записи по типу и формирует итоговый набор данных. В программе предусмотрено ограничение до 200 записей. Фактическое количество строк зависит от числа подходящих утверждений, обнаруженных в исходных публикациях.

Метод `save_dataset_outputs()` сохраняет результаты в нескольких форматах:

- `dataset.json` — основной структурированный набор записей;
- `dataset_50.csv` — табличная выборка до 50 записей;
- `Dataset.xlsx` — Excel-представление до 30 записей;
- `annotation_pairs.csv` — файл для ручной разметки двумя аннотаторами.

Excel-файл дополнительно оформляется: для столбцов задаётся ширина, а для ячеек включается перенос текста. Перед сохранением текст очищается от управляющих символов, которые могли попасть из PDF и привести к ошибке создания файла.

Файл `annotation_pairs.csv` содержит идентификатор записи, исходное утверждение, автоматически назначенный тип и два пустых столбца: `annotator_1` и `annotator_2`. Эти столбцы предназначены для независимой ручной проверки классификации двумя участниками.

4.10 Расчёт согласия аннотаторов

Для оценки согласованности ручной разметки реализован класс `AnnotationAgreementCalculator`.

Метод `cohen_kappa()` вычисляет коэффициент Cohen's kappa. Данный показатель учитывает не только фактическое совпадение ответов двух аннотаторов, но и вероятность случайного совпадения. Значение коэффициента, близкое к 1, означает высокое согласие разметчиков.

Метод `calculate_from_file()` читает заполненный файл `annotation_pairs.csv`, извлекает значения из столбцов `annotator_1` и `annotator_2`, вычисляет коэффициент и сохраняет результат в файл `annotation_agreement.json`.

При первом запуске программы поля аннотаторов являются пустыми, поэтому коэффициент ещё не рассчитывается содержательно. Для получения оценки качества классификации необходимо сначала заполнить файл разметки вручную, а затем повторно выполнить расчёт.

5 ПОРЯДОК РАБОТЫ ПРОГРАММЫ

Для выполнения программы используется следующая структура проекта:

```
project/
├─ main.py
├─ pdfs/
│   └─ article1.pdf
│   └─ article2.pdf
│   └─ ...
└─ output/
```

В каталог `pdfs` помещаются исходные математические публикации в формате PDF. Программа обрабатывает все обнаруженные файлы с расширениями `.pdf` и `.PDF`, включая файлы во вложенных каталогах.

После запуска выполняются следующие этапы:

1. Определяются пути к входным и выходным каталогам.
2. Из PDF-файлов извлекаются текстовые блоки для построения глоссария и таблицы обозначений.
3. Полные тексты статей очищаются и сохраняются в формате TXT.
4. По очищенным текстам рассчитывается статистика терминов.
5. Из текстов выделяются предложения-кандидаты и соседний контекст.

6. Найденные записи классифицируются по типам.
7. Для каждой записи формируются все поля датасета.
8. Создаются итоговые файлы JSON, CSV, XLSX и файл для аннотации.
9. При наличии заполненной ручной разметки рассчитывается Cohen's kappa.
10. Сводная информация о работе программы сохраняется в `report.json`.

6 ОЦЕНКА КАЧЕСТВА РАБОТЫ

Качество работы программы может оцениваться по нескольким направлениям.

Первым критерием является корректность извлечения текста из PDF. Для этого необходимо проверить файлы из каталога `txt_clean` и убедиться, что в них отсутствуют заголовочные данные, списки литературы, разрывы слов и критические артефакты кодировки.

Вторым критерием является релевантность найденных утверждений. Необходимо вручную просмотреть часть записей из `Dataset.xlsx` или `dataset_50.csv` и определить, действительно ли они относятся к алгебраическим структурам и содержат математический смысл.

Третьим критерием является качество автоматической классификации. Для этого два аннотатора независимо заполняют поля `annotator_1` и `annotator_2` в файле `annotation_pairs.csv`. После этого рассчитывается коэффициент Cohen's kappa. В соответствии с постановкой задачи целевым результатом считается значение не ниже 0,70.

Четвёртым критерием является полнота формируемой структуры данных. Каждая строка датасета должна содержать русский фрагмент, англоязычную заготовку, формальное представление, Lean-шаблон, ключевые слова, тип утверждения и источник.

Пятым критерием является техническая устойчивость программы. Она должна корректно обрабатывать несколько PDF-файлов, создавать все необходимые результаты и не завершаться ошибкой при наличии управляющих символов, извлечённых из PDF.

При этом следует учитывать ограничения разработанной реализации. Англоязычное поле создаётся с помощью словарных замен и требует ручной редакторской проверки. Формальные выражения и Lean-код создаются на уровне тематических шаблонов и также требуют последующей экспертной верификации. Поэтому сформированный набор следует рассматривать как подготовленный черновой корпус для дальнейшей разметки и уточнения.

7 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ РАБОТЫ

В результате выполнения работы разработана программа обработки математических PDF-публикаций по теме алгебраических структур. Программа поддерживает пакетную обработку нескольких документов, извлекает и очищает текст, ищет тематически значимые утверждения и преобразует их в структурированный набор данных.

В программе реализованы следующие функциональные возможности:

- формирование глоссария основных алгебраических понятий;
- поиск и подсчёт характерных математических обозначений;
- сохранение очищенных текстовых версий статей;
- расчёт статистики употребления тематических терминов;
- выделение математических утверждений с сохранением контекста;
- удаление повторяющихся фрагментов;
- автоматическая классификация записей по пяти смысловым категориям;
- подготовка полей для параллельного русско-английского представления;
- генерация формальных шаблонов и заготовок Lean 4 + Mathlib;
- сохранение датасета в форматах JSON, CSV и XLSX;
- формирование файла для ручной аннотации;
- расчёт коэффициента согласия аннотаторов.

Выходные данные программы могут использоваться как основа для последующей ручной верификации и подготовки русскоязычного обучающего набора в области автоформализации математики.

ВЫВОД

В ходе работы была разработана программа для подготовки структурированного датасета математических утверждений, извлекаемых из русскоязычных PDF-публикаций. Основное внимание уделено тематике алгебраических структур: групп, колец, полей, модулей, идеалов, гомоморфизмов и изоморфизмов.

Программа объединяет несколько этапов обработки: извлечение текста из PDF, его очистку, поиск ключевых терминов, выделение утверждений, сохранение контекста, классификацию записей и экспорт результатов в удобные форматы. Дополнительно предусмотрены формальные шаблоны, заготовки Lean 4 + Mathlib и механизм ручной аннотации с последующим расчётом Cohen's kappa.

Разработанное решение позволяет автоматизировать начальный этап подготовки русскоязычного математического корпуса. При этом полученные англоязычные формулировки и формальные записи являются предварительными и должны проверяться вручную. Таким образом, результат работы представляет собой рабочую основу для дальнейшего улучшения качества разметки и формирования полноценного обучающего датасета для задач автоформализации математики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Python 3 Documentation [Электронный ресурс]. — URL: <https://docs.python.org/3/> (дата обращения: 20.04.2026).
2. PyMuPDF Documentation. Text Extraction [Электронный ресурс]. — URL: <https://pymupdf.readthedocs.io/en/latest/recipes-text.html> (дата обращения: 24.04.2026).
3. pandas Documentation. DataFrame.to_excel [Электронный ресурс]. — URL: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_excel.html (дата обращения: 25.04.2026).

4. openpyxl Documentation [Электронный ресурс]. — URL: <https://openpyxl.readthedocs.io/> (дата обращения: 25.04.2026).
5. Mathlib Documentation for Lean 4 [Электронный ресурс]. — URL: https://leanprover-community.github.io/mathlib4_docs/index.html (дата обращения: 26.04.2026).